MCMP117 · Week 4 · Course Notes

Fundamentals of Operating Systems

Threads & CPU Scheduling Basics

How can one program do several things at once — and who decides what runs next?

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. Define a *thread* and list what the threads of one process share and what each keeps private.
2. Explain the four benefits of multithreading and the difference between *concurrency* and *parallelism*.
3. Apply Amdahl's law to compute the maximum speedup of a program on N cores.
4. Compare user-level and kernel-level threads and the many-to-one, one-to-one and many-to-many models.
5. Read and run small Pthreads programs in C, and observe threads with `ps` and `/proc`.
6. Describe the CPU–I/O burst cycle, the ready queue, the four scheduling decision points, preemption and the dispatcher.
7. Compute CPU utilisation, throughput, turnaround, waiting and response time from a Gantt chart.

Contents

1 From Processes to Threads **1**

1.1 A short recap of Week 3 . . . 1
1.2 Why threads? . . . 2
1.3 What is a thread? . . . 2
1.4 What is shared, what is private? . . . 3
1.5 Benefits of multithreading . . . 3

2 Multicore Programming **4**

2.1 Concurrency versus parallelism . . . 4
2.2 Programming challenges . . . 4
2.3 Amdahl's law . . . 5

3 User Threads, Kernel Threads and Multithreading Models 6

3.1 User-level and kernel-level threads . . . 6

3.2 The three models . . . 7

4 Thread Libraries and Hands-on Pthreads 7

4.1 Example 1: is a thread really cheaper than a process? . . . 8

4.2 Example 2: hello from three threads . . . 9

4.3 Example 3: summing an array with T threads . . . 10

5 CPU Scheduling: Basic Concepts 12

5.1 The CPU–I/O burst cycle . . . 12

5.2 Example 4: CPU-bound versus I/O-bound, measured . . . 12

5.3 The CPU scheduler and the ready queue . . . 13

5.4 When does the scheduler have to decide? . . . 14

5.5 Non-preemptive and preemptive scheduling . . . 14

5.6 The dispatcher . . . 14

6 Scheduling Criteria 15

6.1 The definitions as formulas . . . 16

6.2 Worked example: four processes in arrival order . . . 17

6.3 Optimisation goals . . . 17

7 Summary 18

8 Glossary 18

9 Try It Yourself 19

10 Review Questions 19

11 Looking Ahead: Week 5 21

1 From Processes to Threads

1.1 A short recap of Week 3

Last week we met the **process**: a program in execution, with a memory image (text, data, bss, heap, stack), a CPU state (program counter, registers) and resources such as open files, all described by its **process control block** (PCB). A process moves through five states (new, ready, running, waiting, terminated); a **context switch** saves the CPU state of one process into its PCB and loads another; `fork()`, `exec()` and

`wait()` create, replace and collect processes.

In that picture every process does exactly one thing at a time: it has *one* program counter. This week we remove that restriction.

1.2 Why threads?

Consider a web server that receives thousands of requests per second. Serving one request needs a little CPU time and a lot of waiting: for the disk, for a database, for the network.

- A **single-threaded** server handles one request at a time. While request 1 waits for the disk, requests 2, 3, 4 wait in line, although the CPU is idle.
- A server could **create a new process** for every request (early Apache web servers did this with `fork()`). This works, but creating a process is expensive, and processes do not share memory, so they cannot easily share a cache of web pages.
- A **multithreaded** server has one *listener* thread that accepts requests and hands each one to a *worker* thread. All threads live inside one process and share its memory.



Figure 1: A single-threaded server serves requests one by one; a multithreaded server gives each request its own thread.

The same idea appears in almost every interactive program. A word processor may have one thread that reads keystrokes, one that redraws the screen, one that checks spelling and one that saves a backup copy every few minutes. A browser loads pages, decodes images and runs scripts in many threads; a game runs rendering, physics, sound and network code in separate threads. In each case the threads work on the *same* data (document, page, game world), so they must share memory.

1.3 What is a thread?

Definition

A **thread** is a basic unit of CPU utilisation. It consists of a *thread ID*, a *program counter*, a *register set* and a *stack*. It shares with the other threads of the same process the code section, the data section and the operating-system resources such as open files.

A traditional, "heavyweight" process is simply a process with exactly one thread. A multithreaded process has several program counters: several places in the same program are being executed at the same time.

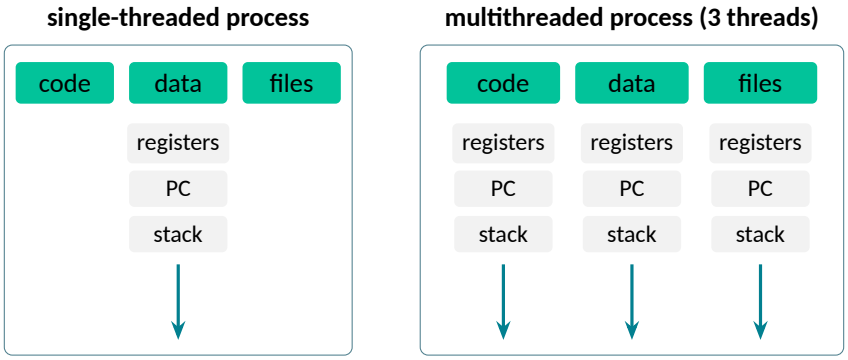


Figure 2: Mint: shared by all threads of the process. Grey: one private copy per thread. Each arrow is one thread of execution.

In terms of Week 3, the PCB is split in two: a per-process part (memory map, open files, user ID) and a small per-thread part (state, saved registers and program counter, stack pointer). Linux calls every thread a *task* and gives it its own kernel record; the threads of one process simply point to the same memory map and the same table of open files. Older texts call threads **lightweight processes** (LWP); the abbreviation still appears in the output of `ps`.

1.4 What is shared, what is private?

Item	Threads...	Why
Code (text section)	share	All threads run functions of the same program.
Global data and heap	share	A global variable or a `malloc`'ed array is visible to every thread.
Open files, PID, user ID	share	Resources belong to the process, not to a thread.
Program counter and registers	private	Each thread is at its own place in the code.
Stack (local variables)	private	Each thread has its own chain of function calls.
Thread ID, scheduling state	private	The OS schedules each thread separately.

Sharing is the whole point of threads — and also their main danger. If two threads update the same variable at the same moment, updates can be lost. This is a **race condition**, the central topic of the second half of Week 5.

1.5 Benefits of multithreading

1. **Responsiveness.** If one thread blocks (waits for I/O) or performs a long computation, the other threads keep running. The familiar "application not responding" message usually means that the single thread handling the user interface is busy with something else.
2. **Resource sharing.** Threads share the memory and resources of their process automatically. Separate processes must use explicit mechanisms such as shared memory or pipes to cooperate.
3. **Economy.** Creating a thread and switching between threads of the same process are much cheaper

than for processes, because no new address space has to be created or switched. We measure this in Section 4.1.

4. **Scalability.** On a multicore processor, the threads of one process can run in parallel on different cores. A single-threaded process can use only one core, however many the machine has.

> **Why threads became essential**
>
> Until about 2005, every new generation of processors ran at a higher clock frequency, so old single-threaded programs became faster automatically. Then power consumption and heat stopped the growth of clock speeds at roughly 3–5 GHz. Chip makers used the extra transistors to put several cores on one chip instead. Since then, a program becomes faster on new hardware only if it can use several cores — that is, only if it is multithreaded (or split into several processes).

2 Multicore Programming

2.1 Concurrency versus parallelism

These two words are often confused.

- A system is **concurrent** if it supports more than one task *making progress*. On a single core, the OS achieves this by switching between threads every few milliseconds; to a human, the threads appear to run simultaneously.
- A system is **parallel** if it can perform more than one task *at the same instant*. This needs at least two cores.



Figure 3: Four threads on one core (concurrency without parallelism) and on two cores (concurrency with parallelism).

Parallelism implies concurrency, but not the other way round. The test machine used for this week's measurements has two cores (`nproc` prints 2).

2.2 Programming challenges

Writing programs that use several cores well is hard. Five challenges are usually listed:

1. **Identifying tasks:** finding parts of the work that can run independently.
2. **Balance:** giving each core a similar amount of work; otherwise some cores sit idle while one finishes.
3. **Data splitting:** dividing the data so that each thread works on its own part.

4. **Data dependency:** if task B needs the result of task A, B must wait for A. The threads must be *synchronised* (Week 5).
5. **Testing and debugging:** the threads can interleave in very many different orders; a bug may appear only once in a million runs.

There are two basic kinds of parallelism. In **data parallelism** the same operation is applied to different parts of the data; our array-sum program (Section 4.3) is an example. In **task parallelism** different threads perform different operations, like the word-processor threads above. Real programs often mix both.

2.3 Amdahl's law

Almost every program has a part that cannot be split among cores: reading the input, combining partial results, printing the answer. Gene Amdahl (IBM) observed in 1967 that this *serial* part limits the speedup that any number of cores can deliver.

Let the running time on one core be 1 (we normalise it), and let S be the fraction of that time that is serial, $0 \le S \le 1$. The remaining fraction $1 - S$ can be divided perfectly among N cores. Then

$$T(1) = S + (1 - S) = 1, \qquad T(N) = S + \frac{1 - S}{N}.$$

The **speedup** is the ratio of the old time to the new time:

Amdahl's law

$$\text{speedup}(N) = \frac{T(1)}{T(N)} \le \frac{1}{S + \dfrac{1 - S}{N}}, \qquad \lim_{N \to \infty} \text{speedup}(N) = \frac{1}{S}.$$

The inequality sign reminds us that this is a best case: real programs also pay for creating threads, synchronising them and sharing memory bandwidth.

75 % parallel, 25 % serial

Take $S = 0.25$, so $1 - S = 0.75$.

$$\begin{aligned}
N = 2: &\quad \frac{1}{0.25 + 0.75/2} = \frac{1}{0.625} = 1.60, \\
N = 4: &\quad \frac{1}{0.25 + 0.75/4} = \frac{1}{0.4375} \approx 2.29, \\
N = 8: &\quad \frac{1}{0.25 + 0.75/8} = \frac{1}{0.34375} \approx 2.91, \\
N = 16: &\quad \frac{1}{0.25 + 0.75/16} = \frac{1}{0.296875} \approx 3.37, \\
N \to \infty: &\quad \frac{1}{0.25} = 4.
\end{aligned}$$

Doubling the hardware from 8 to 16 cores raises the speedup only from 2.91 to 3.37. No number of cores can make this program more than 4 times faster.

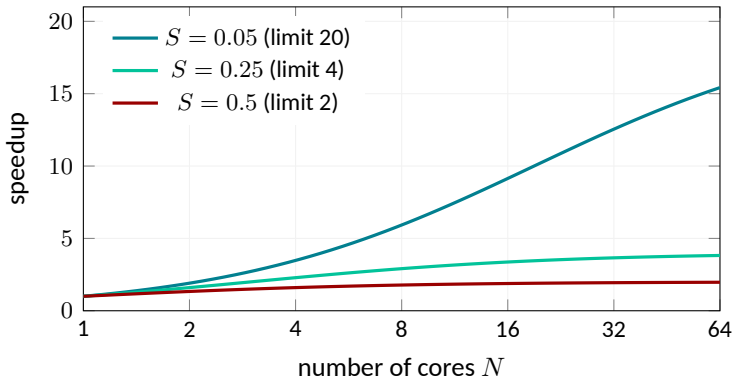


Figure 4: Amdahl's law: the speedup flattens out at $1/S$.

The practical lesson: *first make the serial part small, then add cores.* This is why programs for graphics processors, which have thousands of cores, are designed to keep the serial part tiny.

3 User Threads, Kernel Threads and Multithreading Models

3.1 User-level and kernel-level threads

Threads can be provided at two levels.

User-level threads are managed by a library inside the process. The kernel knows nothing about them: it sees one ordinary single-threaded process.

+ Creating and switching threads are ordinary function calls: very fast, no system call.
− If one thread makes a blocking system call (e.g. `read()` from a slow device), the kernel blocks the whole process, and all its threads stop.
− The kernel schedules only one entity, so the threads cannot run on two cores at once.

Kernel-level threads are created, scheduled and managed by the operating-system kernel; each has its own kernel record.

+ If one thread blocks, the kernel runs another thread of the same process.
+ The threads of one process can run in parallel on several cores.
− Creating and switching threads need system calls and are slower than for user threads.

Every user thread must eventually run on some kernel thread. The way user threads are mapped onto kernel threads is called the **multithreading model**.

3.2 The three models

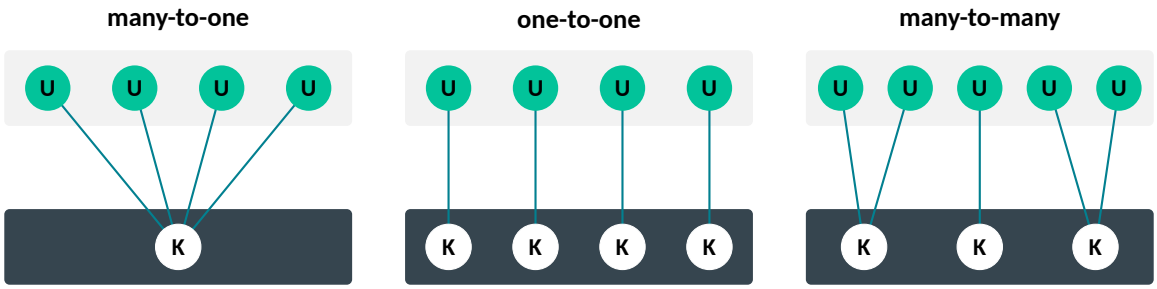


Figure 5: The three multithreading models. Light band: user space with user threads (U); dark band: kernel space with kernel threads (K).

Many-to-one. Many user threads are mapped onto a single kernel thread; thread management is done by a library in user space. It is fast, but one blocking system call blocks every thread, and only one thread can be in the kernel at a time, so the model cannot use multiple cores. Examples: Solaris Green Threads, GNU Portable Threads. Few systems use it today.

One-to-one. Each user thread is mapped onto its own kernel thread. A blocking call blocks only the calling thread, and threads run in parallel on multicore processors. The cost is that every user thread needs a kernel thread, so creating a thread requires a system call and very large numbers of threads burden the kernel. **Linux and Windows use the one-to-one model.** On Linux, `pthread_create()` calls the `clone()` system call, and every thread gets its own thread ID (TID) in the kernel.

Many-to-many. Many user threads are multiplexed onto a smaller or equal number of kernel threads. The program can create as many user threads as it likes, and they can still run in parallel; a blocking call does not stop everything. The price is complexity: a scheduler in the library and the scheduler in the kernel must cooperate. A common variation, the *two-level model*, also allows a user thread to be bound to its own kernel thread. Operating-system kernels abandoned this model for one-to-one, but language runtimes revived it: Go's *goroutines* and Java's *virtual threads* (Java 21, 2023) run millions of lightweight tasks on a few kernel threads.

Model	User : kernel	Parallel?	Blocking call stops...	Used by
Many-to-one	$N : 1$	no	all threads	Green threads (old)
One-to-one	$1 : 1$	yes	only that thread	Linux, Windows
Many-to-many	$N : M, M \leq N$	yes	only that thread	Go, Java virtual threads

4 Thread Libraries and Hands-on Pthreads

A **thread library** gives the programmer an API for creating and managing threads. The three main libraries are **POSIX Pthreads** (Linux, macOS and other UNIX systems), the **Windows thread API**, and **Java threads**. On Linux and Windows, Java threads are mapped one-to-one onto kernel threads by the Java virtual machine.

Pthreads is a *specification* (part of the POSIX standard), not an implementation. On Linux it is implemented in the C library (glibc, under the name NPTL) on top of the `clone()` system call. The essential calls are:

Call	What it does
`pthread_create(&t, NULL, f, arg)`	Start a new thread that runs `f(arg)`; its handle is stored in `t`.
`pthread_join(t, &result)`	Wait until thread `t` has finished (like `wait()` for processes).
`pthread_exit(value)`	End the calling thread (returning from `f` does the same).
`pthread_self()`	Return the calling thread's own handle.
`gcc -pthread prog.c`	Compile and link with the Pthreads library.

A thread function always has the form `void *f(void *arg)`: it receives one generic pointer and returns one. To pass several values, pass a pointer to a `struct`.

The programs below are in the `code/` folder next to these notes, with a `Makefile` (type `make`). They were compiled with `gcc -Wall -O2 -pthread` and run on a Linux virtual machine with two cores that was shared with other jobs. As in Week 2, you are not expected to write C in this course: read the comments and focus on what the program does.

4.1 Example 1: is a thread really cheaper than a process?

```c
#define N 5000
void *nothing(void *arg) { return arg; }

/* 1) N processes */
for (int i = 0; i < N; i++) {
    pid_t pid = fork();           /* copy the process */
    if (pid == 0) _exit(0);       /* child exits      */
    waitpid(pid, NULL, 0);
}
/* 2) N threads */
for (int i = 0; i < N; i++) {
    pthread_t t;
    pthread_create(&t, NULL, nothing, NULL);
    pthread_join(t, NULL);
}
/* full program: code/create_cost.c */
```

```
$ ./create_cost
fork + wait          :  205.5 us per process
pthread_create + join:   51.9 us per thread
a thread is about 4 times cheaper to create
```

(`us` means microseconds, μs.) `fork()` must create a new address space: copy the page tables and many kernel structures, even though the memory pages themselves are shared copy-on-write until someone writes to them. A new thread needs only a stack and a small kernel record. On a fast physical machine

thread creation often takes only 10–20 μs and the ratio is larger; textbooks quote factors of 10 to 30. Our ratio of about 4 comes from a virtual machine, where some operations are slower.

4.2 Example 2: hello from three threads

```c
/* thread_hello.c -- one process, several threads.
 * Every thread prints the process ID (PID, shared by all threads)
 * and its own thread ID (TID, private to each thread).
 * Compile:  gcc -Wall -O2 -pthread thread_hello.c -o thread_hello
 * Run:      ./thread_hello
 */
#define _GNU_SOURCE
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>        /* getpid(), gettid(), sleep() */

#define NTHREADS 3

int shared_counter = 0;    /* global: ONE copy, seen by all threads */

/* Every thread starts its life in this function. */
void *worker(void *arg)
{
    int my_number = *(int *)arg;   /* local: on this thread's OWN stack */
    printf("thread %d: PID = %d, TID = %d\n",
           my_number, getpid(), gettid());
    shared_counter++;              /* (unsafe! see Week 5) */
    sleep(2);                      /* stay alive so we can watch it with ps */
    return NULL;
}

int main(void)
{
    pthread_t tid[NTHREADS];       /* thread "handles"            */
    int number[NTHREADS];

    printf("main    : PID = %d, TID = %d\n", getpid(), gettid());
    for (int i = 0; i < NTHREADS; i++) {
        number[i] = i + 1;
        /* create a thread that runs worker(&number[i]) */
        pthread_create(&tid[i], NULL, worker, &number[i]);
    }
    for (int i = 0; i < NTHREADS; i++)
        pthread_join(tid[i], NULL);  /* wait for thread i to finish */

    printf("main    : all threads done, shared_counter = %d\n",
           shared_counter);
    return 0;
}
```

```
$ gcc -Wall -O2 -pthread thread_hello.c -o thread_hello
```

```
$ ./thread_hello
main    : PID = 6351, TID = 6351
thread 1: PID = 6351, TID = 6352
thread 3: PID = 6351, TID = 6354
thread 2: PID = 6351, TID = 6353
main    : all threads done, shared_counter = 3
```

Three observations:

- All threads print the *same* PID: they belong to one process. Each has its *own* TID, because each is a separate kernel thread (one-to-one model). The main thread's TID equals the PID.
- Thread 3 printed before thread 2. The order is decided by the scheduler and changes from run to run; a program must never rely on it.
- `shared_counter` ends at 3: the global variable is shared. But `shared_counter++` is not safe when threads run at the same time (it is really "load, add, store"). With three increments the bug almost never shows; with millions it does. We fix this in Week 5.

Seeing the threads. Each worker sleeps for two seconds, which gives us time to look at the running process. The option `-L` of `ps` prints one line per thread (some columns removed):

```
$ ./thread_hello &
$ ps -eLf | grep thread_hello
UID    PID  PPID   LWP  C NLWP CMD
root 27241 27234 27241  0    4 ./thread_hello
root 27241 27234 27243  0    4 ./thread_hello
root 27241 27234 27244  0    4 ./thread_hello
root 27241 27234 27245  0    4 ./thread_hello
$ ls /proc/27241/task
27241  27243  27244  27245
$ grep Threads /proc/27241/status
Threads:   4
```

The PID is the same on every line; `LWP` (lightweight process) is the thread ID and `NLWP` the number of threads: the main thread plus three workers. The directory `/proc/<pid>/task/` contains one subdirectory per thread. Other tools: `top -H` or `htop` show threads interactively; on Windows, Task Manager → *Details* can show a *Threads* column.

4.3 Example 3: summing an array with T threads

```
1  int  *a;              /* the big array: shared  */
2  long  partial[MAXT];  /* one slot per thread    */
3
4  void *sum_slice(void *arg)
5  {
6      long k = (long)arg;              /* my slice */
7      long start = k * N / T, end = (k + 1) * N / T;
8      long s = 0;
9      for (long i = start; i < end; i++)
10         s += a[i];
```

```
    partial[k] = s;                   /* my slot  */
    return NULL;
}

/* in main(): */
for (long k = 0; k < T; k++)
    pthread_create(&th[k], NULL, sum_slice, (void *)k);
for (long k = 0; k < T; k++) {
    pthread_join(th[k], NULL);
    total += partial[k];
}
/* full program (adds timing, 20 repeats): code/array_sum.c */
```

The array of $N = 10^8$ integers (400 MB) is cut into T equal slices. Thread k sums the elements with indices from kN/T up to (but not including) $(k+1)N/T$ into its own slot `partial[k]`. Because no two threads write to the same variable, no locks are needed. The main thread waits for every thread with `pthread_join()` and adds the partial sums. The full program `code/array_sum.c` also measures the time with `clock_gettime()` and sums the array 20 times, so that the measured time is long enough to be stable; the correct result is $10^7 \times (0+1+\cdots+9) \times 20 = 10^7 \times 45 \times 20 = 9 \times 10^9$, which every run reproduced.

```
$ nproc
2
$ ./array_sum 1
threads = 1   sum = 9000000000   time = 1.686 s
$ ./array_sum 2
threads = 2   sum = 9000000000   time = 0.999 s
$ ./array_sum 4
threads = 4   sum = 9000000000   time = 0.855 s
```

Because the machine was shared with other jobs, single runs varied by up to about $\pm 30\%$. We therefore ran each case 9 times and report the median:

Threads T	Median time	Best time	Speedup (median)
1	1.564 s	1.498 s	1.00
2	0.932 s	0.761 s	$1.564/0.932 \approx 1.68$
4	0.855 s	0.728 s	$1.564/0.855 \approx 1.83$

Interpretation. Two threads give a speedup of 1.68, good but below the ideal value 2. Four threads are hardly better, because the machine has only two cores: at any instant at most two threads can run. (On a busy machine four threads can be slightly faster than two simply because the process then receives a larger share of the CPU time, but never faster than the number of cores allows.) If we pretend that Amdahl's law describes the measurement, we can solve for S:

$$1.68 = \frac{1}{S + (1-S)/2} \implies S + \frac{1-S}{2} = \frac{1}{1.68} \implies S = \frac{2}{1.68} - 1 \approx 0.19.$$

Here the "serial part" is not really code that cannot be split: it is the time to create and join threads, the competition of both cores for the same memory bandwidth (summing integers is limited by memory speed,

not arithmetic), and the other jobs on the machine. Amdahl's law is a model and an upper bound, not an exact prediction.

5 CPU Scheduling: Basic Concepts

On a machine with one core, only one thread can run at a time; with N cores, at most N. All other runnable threads must wait. Whenever a core becomes free, the operating system must choose which thread runs next. This choice is **CPU scheduling**. Textbooks speak of "process scheduling"; on Linux and Windows the kernel actually schedules kernel threads, but everything below applies to both, and we follow the textbook in saying "process".

5.1 The CPU--I/O burst cycle

Process execution consists of a *cycle* of CPU execution and I/O waiting. A process runs for a while (a **CPU burst**), then waits for I/O (an **I/O burst**), then runs again, and so on, until a final CPU burst ends with a request to terminate.

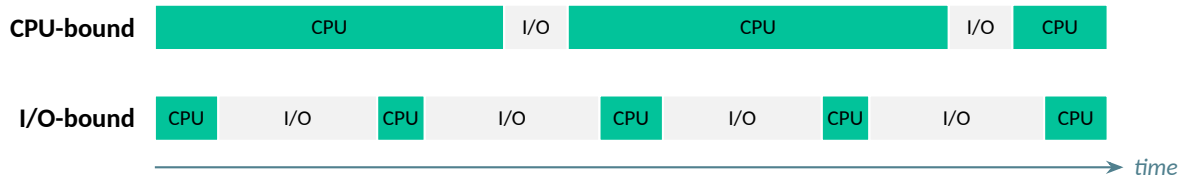


Figure 6: A CPU-bound process has few, long CPU bursts; an I/O-bound process has many short ones.

- A **CPU-bound** process spends most of its time computing: video encoding, compiling, scientific simulation.
- An **I/O-bound** process spends most of its time waiting for I/O and has many short CPU bursts: a text editor, a web server, a file copy.

Measurements of real systems show that the lengths of CPU bursts follow a typical pattern: a large number of short bursts (a few milliseconds) and a small number of long ones. The distribution is roughly exponential. Scheduling algorithms are designed around this fact (Week 5). While a process waits for I/O, the CPU would be idle — unless the scheduler gives it to another process: the idea of multiprogramming from Week 1.

5.2 Example 4: CPU-bound versus I/O-bound, measured

```c
/* ./burst cpu : one long CPU burst */
for (long i = 1; i <= 400000000L; i++)
    x += 1.0 / i;

/* ./burst io : 1000 x (tiny CPU burst + I/O wait) */
for (int i = 0; i < 1000; i++) {
    buf[0] = 'A' + i % 26;     /* CPU    */
    write(fd, buf, 4096);      /* I/O... */
    fsync(fd);                 /* ...wait for disk */
}
/* full program: code/burst.c */
```

The shell command `time` reports three numbers: `real` is the elapsed wall-clock time; `user` and `sys` are the CPU time the process consumed in user mode and in kernel mode.

```
$ time ./burst cpu
cpu: harmonic sum = 20.384191
real    0m1.266s
user    0m1.197s
sys     0m0.001s
$ time ./burst io
io: wrote 1000 blocks of 4 KB, each followed by fsync
real    0m0.384s
user    0m0.000s
sys     0m0.104s
```

How much of the time was the process waiting?

For the CPU-bound run, user + sys $= 1.198$ s $\approx$ real $= 1.266$ s: the process used the CPU almost all the time. For the I/O-bound run,

$$\frac{\text{user} + \text{sys}}{\text{real}} = \frac{0.000 + 0.104}{0.384} \approx 0.27,$$

so the process used the CPU only 27% of the time; about 73% of the time it was blocked in `fsync()`, waiting for the disk. During that time the CPU was free for other processes. Note also that almost all of its CPU time is `sys`: `write()` and `fsync()` are system calls. (The harmonic sum checks out: $\sum_{i=1}^{n} 1/i \approx \ln n + 0.5772$, and $\ln(4 \times 10^8) + 0.5772 \approx 20.384$.)

5.3 The CPU scheduler and the ready queue

Whenever the CPU becomes idle, the operating system selects one of the processes in the **ready queue** to run. The part of the OS that makes this choice is the **short-term scheduler** or **CPU scheduler**.

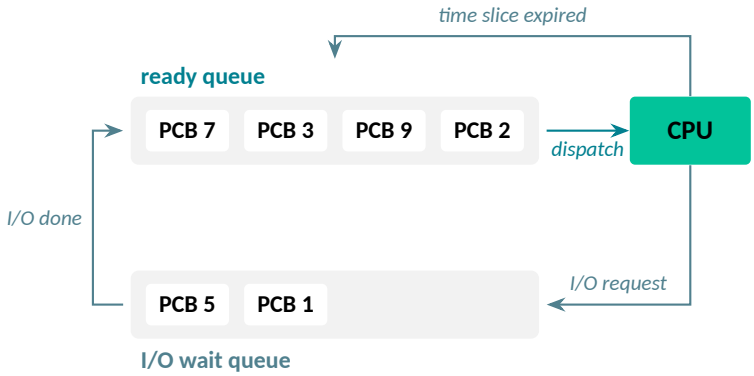


Figure 7: Processes move between the ready queue, the CPU and device wait queues.

- The ready queue holds the PCBs of exactly those processes that are in the *ready* state of the Week 3 diagram: they could run right now.
- The ready queue is not necessarily first-in, first-out. It may be a priority queue, a tree or an unordered list; the order in which processes are taken from it is the **scheduling algorithm** (Week 5). Linux keeps its ready tasks in a balanced binary tree (a red–black tree).

- The short-term scheduler runs very often — every few milliseconds — so it must be fast.

5.4 When does the scheduler have to decide?

Scheduling decisions may take place in four situations, all visible in the process-state diagram:

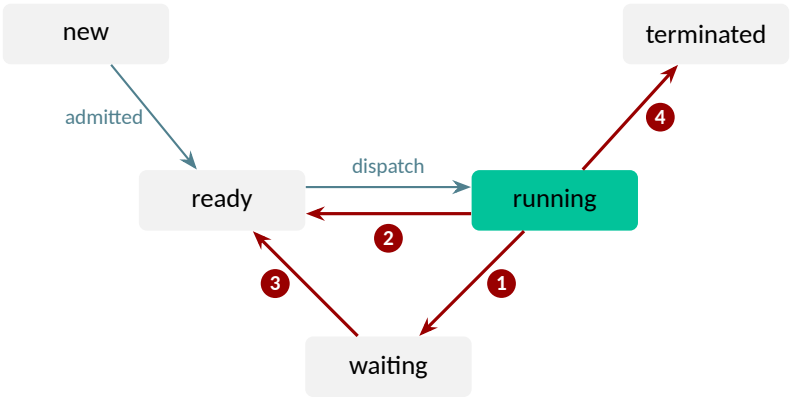


Figure 8: The four transitions at which a scheduling decision can be made.

1. A process switches from *running* to *waiting*: it requests I/O or calls `wait()` for a child.
2. A process switches from *running* to *ready*: an interrupt occurs, for example the timer signals that its time slice is over.
3. A process switches from *waiting* to *ready*: its I/O has completed.
4. A process *terminates*.

In situations 1 and 4 there is no choice: the CPU has become free, and a new process *must* be selected (if the ready queue is not empty). In situations 2 and 3 there is a choice: the OS *may* let the current process continue or switch to another one — for example to a more important process that has just become ready.

5.5 Non-preemptive and preemptive scheduling

- Under **non-preemptive** (or *cooperative*) scheduling, the scheduler runs only in situations 1 and 4. Once a process has the CPU, it keeps it until it blocks or terminates. This is simple, but one long or misbehaving process can make all others wait. Windows 3.x and the classic Mac OS (before OS X) worked this way: a program stuck in an infinite loop froze the whole computer.
- Under **preemptive** scheduling, the scheduler can also run in situations 2 and 3: the OS can take the CPU away from a running process, using the timer interrupt. All modern general-purpose operating systems — Windows, macOS, Linux, Android, iOS — are preemptive.

Preemption gives better responsiveness and protects the system from runaway programs, but it has a price: a process can be interrupted at *any* instruction, even in the middle of updating data shared with another process or thread. This is how race conditions arise, and why Week 5 continues with synchronisation.

5.6 The dispatcher

The scheduler decides *which* process runs next; the **dispatcher** is the module that actually gives it the CPU. Its three jobs are:

1. switch context: save the state of the old process into its PCB and load the state of the new one;
2. switch to user mode;
3. jump to the proper location in the new program (its saved program counter).

The time the dispatcher needs to stop one process and start another is the **dispatch latency**. It is pure overhead: no useful work is done during it.

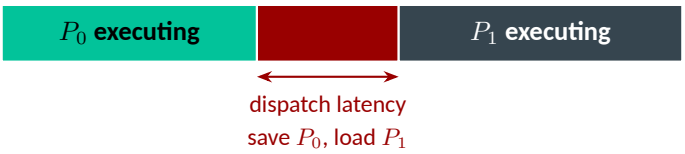


Figure 9: Dispatch latency (not to scale: it is microseconds, a time slice is milliseconds).

Dispatch latency is small, typically a few microseconds, but it happens very often. The command `vmstat 1` prints, among other things, the number of context switches per second in the column `cs`. On our otherwise idle test machine two consecutive one-second samples showed 487 and 614; a busy server easily reaches tens of thousands.

Voluntary and involuntary context switches. Linux counts separately how often a process gave up the CPU itself, because it blocked (a *voluntary* switch, situation 1), and how often the scheduler took the CPU away (an *involuntary* switch, situation 2). The GNU `time` program with option `-v` reports both (other lines, including the user and system times, removed):

```
$ /usr/bin/time -v ./burst cpu
  Elapsed (wall clock) time (h:mm:ss or m:ss): 0:01.22
  Voluntary context switches: 1
  Involuntary context switches: 50
$ /usr/bin/time -v ./burst io
  Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.46
  Voluntary context switches: 3714
  Involuntary context switches: 0
```

The CPU-bound program almost never blocked, but it was preempted 50 times in 1.22 s, on average about once every 24 ms. The I/O-bound program was never preempted but blocked 3714 times (each `fsync()` may wait more than once, e.g. for the file-system journal). The same counters are visible for a running process in `/proc/<pid>/status` (fields `voluntary_ctxt_switches` and `nonvoluntary_ctxt_switches`).

6 Scheduling Criteria

Different scheduling algorithms favour different processes. To compare them we need measurable criteria.

Criterion	Definition	Goal
CPU utilisation	Fraction of time the CPU is busy with useful work; on a real system from about 40% (light load) to 90% (heavy load).	maximise
Throughput	Number of processes completed per unit of time.	maximise
Turnaround time	Time from the arrival (submission) of a process to its completion, including waiting in the ready queue, executing and doing I/O.	minimise
Waiting time	Total time a process spends waiting in the *ready queue*.	minimise
Response time	Time from arrival until the process first gets the CPU (its first response).	minimise

The scheduling algorithm does not change how long a process computes or how long its I/O takes; it only changes how long the process sits in the ready queue. That is why *waiting time* is the criterion most directly affected by the scheduler.

6.1 The definitions as formulas

For a process with arrival time T_{arrival}, total CPU burst time T_{burst}, first time on the CPU $T_{\text{first run}}$ and completion time $T_{\text{completion}}$, and doing no I/O:

$$\begin{aligned} T_{\text{turnaround}} &= T_{\text{completion}} - T_{\text{arrival}}, \\ T_{\text{waiting}} &= T_{\text{turnaround}} - T_{\text{burst}}, \\ T_{\text{response}} &= T_{\text{first run}} - T_{\text{arrival}}. \end{aligned}$$

If the process also performs I/O, the time spent in I/O must be subtracted as well: $T_{\text{waiting}} = T_{\text{turnaround}} - T_{\text{burst}} - T_{\text{I/O}}$, because waiting time counts only the time in the ready queue. For a set of n processes completed in a time interval of length T during which the CPU was busy for B,

$$\text{CPU utilisation} = \frac{B}{T}, \qquad \text{throughput} = \frac{n}{T}.$$



Figure 10: A process that is preempted once: burst $= 3 + 2 = 5$, waiting $= 4 + 2 = 6$, turnaround $= 11$, response $= 4$.

Checking the formulas on the picture

$T_{\text{turnaround}} = 11 - 0 = 11$; $T_{\text{waiting}} = 11 - 5 = 6$, which is indeed $4 + 2$ units in the ready queue; $T_{\text{response}} = 4 - 0 = 4$. For a process like this, response $\leq$ waiting $\leq$ turnaround. Under non-preemptive scheduling a process waits only once, before its single run, so its response time equals its waiting time.

6.2 Worked example: four processes in arrival order

Four processes arrive at a single CPU. We run them in the "obvious" order, first come, first served (FCFS), without preemption. Whether this is a good order is the question of Week 5; here we only practise the definitions. All times are in milliseconds.

Process	Arrival	Burst
P_1	0	5
P_2	1	3
P_3	2	6
P_4	16	2

Building the Gantt chart. At $t = 0$ only P_1 is present and runs. P_2 ($t = 1$) and P_3 ($t = 2$) arrive while P_1 runs and wait in the ready queue. At $t = 5$, P_1 finishes and P_2, which arrived first, runs until $t = 8$; then P_3 runs until $t = 14$. Now the ready queue is empty, so the CPU is idle until P_4 arrives at $t = 16$; P_4 runs until $t = 18$.



Figure 11: Gantt chart of the worked example (FCFS order).

Process	Arrival	Burst	Completion	Turnaround	Waiting	Response
P_1	0	5	5	$5 - 0 = 5$	$5 - 5 = 0$	0
P_2	1	3	8	$8 - 1 = 7$	$7 - 3 = 4$	4
P_3	2	6	14	$14 - 2 = 12$	$12 - 6 = 6$	6
P_4	16	2	18	$18 - 16 = 2$	$2 - 2 = 0$	0
Average				$26/4 = 6.5$	$10/4 = 2.5$	2.5

- Average turnaround time $= (5+7+12+2)/4 = 6.5$ ms; average waiting time $= (0+4+6+0)/4 = 2.5$ ms; average response time $= 2.5$ ms (equal to the waiting time, because the scheduling is non-preemptive).
- CPU utilisation $= 16$ ms busy$/18$ ms $\approx 88.9\%$. The idle gap lowers utilisation but does not increase anybody's waiting time.
- Throughput $= 4/18$ ms ≈ 0.22 processes per ms, i.e. about 222 processes per second.

6.3 Optimisation goals

In summary, we want to **maximise** CPU utilisation and throughput and **minimise** turnaround time, waiting time and response time. Usually the *average* value is optimised. In some cases it is better to optimise the minimum or maximum value; for interactive systems, a *predictable* response time (low variance) can matter more than a low average: a constant 50 ms feels better than 5 ms that occasionally becomes 2 s.

The goals conflict, and different systems weight them differently:

- **Batch systems and servers:** throughput and CPU utilisation.
- **Desktops and phones:** response time and its predictability.
- **Real-time systems:** meeting every deadline matters more than any average.

A typical trade-off: switching between processes more often improves response time, but each switch costs dispatch latency, so throughput drops. This is the efficiency-versus-convenience conflict of Week 2 in a new form.

7 Summary

Week 4 in seven sentences

1. A thread is a unit of execution with its own thread ID, program counter, registers and stack; the threads of a process share its code, data, heap and open files.
2. Multithreading brings responsiveness, resource sharing, economy (a thread was about 4 times cheaper to create than a process on our machine) and scalability on multicore processors.
3. Concurrency means several tasks make progress; parallelism means several tasks run at the same instant, which needs several cores.
4. Amdahl's law, speedup $\leq 1/\big(S+(1-S)/N\big)$, shows that the serial fraction S limits the speedup to $1/S$.
5. User threads are mapped onto kernel threads many-to-one, one-to-one (Linux, Windows) or many-to-many; Pthreads offers `pthread_create()` and `pthread_join()`.
6. Processes alternate CPU and I/O bursts; the short-term scheduler picks the next process from the ready queue at four kinds of events, and preemptive schedulers may also interrupt a running process; the dispatcher performs the switch.
7. Schedulers are judged by CPU utilisation and throughput (maximise) and by turnaround, waiting and response time (minimise).

8 Glossary

Amdahl's law An upper bound on the speedup of a program with serial fraction S on N cores: $1/(S + (1 - S)/N)$.

Concurrency Several tasks making progress over the same period, possibly by taking turns on one core.

CPU burst / I/O burst A period in which a process computes / waits for I/O.

CPU-bound, I/O-bound A process that spends most of its time computing / waiting for I/O.

Dispatcher The OS module that gives the CPU to the process chosen by the scheduler.

Dispatch latency The time the dispatcher needs to stop one process and start another.

Kernel-level thread A thread created and scheduled by the OS kernel.

Multithreading model The mapping of user threads onto kernel threads: many-to-one, one-to-one or many-to-many.

Parallelism Several tasks executing at the same instant on different cores.

Preemptive scheduling Scheduling in which the OS can take the CPU away from a running process.

Pthreads The POSIX standard API for creating and managing threads.

Ready queue The set of processes that are ready to run and waiting for a CPU.

Response time Time from arrival until a process first gets the CPU.

Short-term (CPU) scheduler The part of the OS that selects the next process to run from the ready queue.

Thread A unit of CPU execution: thread ID, program counter, registers and stack, sharing the rest of its process.

Throughput Number of processes completed per unit of time.

Turnaround time Completion time minus arrival time.

User-level thread A thread managed by a library in user space, invisible to the kernel.

Waiting time Total time a process spends in the ready queue.

9 Try It Yourself

Threads and scheduling on your own machine

Linux (lab PCs, WSL, or a virtual machine), inside the `code/` folder:

```
$ nproc                          # how many cores?
$ make                           # build all four programs
$ ./array_sum 1; ./array_sum 2; ./array_sum 4
$ ./thread_hello & ps -eLf | grep thread_hello
$ time ./burst cpu ; time ./burst io
$ vmstat 1 5                     # column cs = context switches per second
```

Windows: in Task Manager → *Details*, right-click a column header, choose *Select columns* and add *Threads*. Task Manager → *Performance* → *CPU* shows the number of cores and of logical processors.

1. How many cores does your machine have? What speedup do you get with 2, 4 and 8 threads?
2. Estimate S from your two-thread speedup using Amdahl's law.
3. Which program on your computer has the most threads?
4. Is your browser CPU-bound or I/O-bound while you read a page? And while it plays a video?

10 Review Questions

Conceptual

1. Define a thread. Which parts of a process are shared by its threads, and which are private to each thread?
2. Give the four benefits of multithreading, with one example each.
3. Explain the difference between concurrency and parallelism. Can a single-core computer be concurrent? Parallel?
4. Name the five challenges of multicore programming. Which of them does the array-sum program avoid, and how?
5. Compare user-level and kernel-level threads: one advantage and one disadvantage of each.
6. Why can a many-to-one thread library not use a second core? Which model do Linux and Windows use?
7. In `thread_hello`, why do all threads print the same PID but different TIDs? Why can the order of the lines change?
8. Describe the CPU–I/O burst cycle. How do CPU-bound and I/O-bound processes differ, and how does `time` reveal the difference?
9. List the four situations in which a scheduling decision may be made. In which of them *must* a new process be chosen?
10. What is the difference between preemptive and non-preemptive scheduling? Why does preemption lead to race conditions?
11. What does the dispatcher do? What is dispatch latency, and why should it be small?
12. Why is waiting time the criterion most directly influenced by the scheduling algorithm?

Numerical

13. A program is 60% parallel ($S = 0.4$). Compute the maximum speedup on 2, 4 and 8 cores, and the limit for infinitely many cores.
14. A program runs for 100 s on one core; 80 s of this can be parallelised perfectly. How long does it run on 8 cores, and what is the speedup?
15. You need a speedup of at least 3 on 4 cores. What is the largest serial fraction S that allows this?
16. A web server receives 2000 requests per second. Using our measured costs (205.5 μs to create a process, 51.9 μs to create a thread), what fraction of one core is spent only on creating a process, or a thread, for every request?
17. Four processes run FCFS without preemption: A (arrival 0, burst 8), B (2, 4), C (4, 1), D (6, 3), times in ms. Draw the Gantt chart and compute each process's turnaround and waiting time, the averages, the CPU utilisation and the throughput.
18. A context switch takes 5 μs. What fraction of the CPU time is overhead if the time slice is (a) 10 ms, (b) 1 ms? (Assume a switch after every time slice.)

Answers to the numerical questions

13. $N = 2$: $1/(0.4 + 0.3) \approx 1.43$; $N = 4$: $1/(0.4 + 0.15) \approx 1.82$; $N = 8$: $1/(0.4 + 0.075) \approx 2.11$; limit $1/0.4 = 2.5$.

14. $T(8) = 20 + 80/8 = 30$ s; speedup $= 100/30 \approx 3.33$. (Here $S = 0.2$: $1/(0.2 + 0.8/8) = 1/0.3 \approx 3.33$.)

15. $3 = 1/\big(S+(1-S)/4\big) \Rightarrow S+(1-S)/4 = 1/3 \Rightarrow 4S+1-S = 4/3 \Rightarrow 3S = 1/3 \Rightarrow S = 1/9 \approx 0.11$, i.e. at most about 11% serial.

16. Processes: $2000 \times 205.5 \times 10^{-6}$ s $= 0.411$ s of CPU per second, i.e. 41.1% of a core. Threads: $2000 \times 51.9 \times 10^{-6} \approx 0.104$ s, i.e. 10.4%. (Real servers avoid even this by reusing threads from a *thread pool*.)

17. Gantt chart: A 0–8, B 8–12, C 12–13, D 13–16. Turnaround: $A = 8$, $B = 12-2 = 10$, $C = 13-4 = 9$, $D = 16 - 6 = 10$; average $37/4 = 9.25$ ms. Waiting: $A = 0$, $B = 10 - 4 = 6$, $C = 9 - 1 = 8$, $D = 10 - 3 = 7$; average $21/4 = 5.25$ ms. The CPU is never idle: utilisation 100%; throughput $4/16 = 0.25$ processes per ms. (Notice that the short process C waits 8 ms for a 1 ms burst — a hint for Week 5.)

18. (a) $5/(10\,000 + 5) \approx 0.05\%$; (b) $5/(1000 + 5) \approx 0.50\%$. A ten times shorter time slice costs ten times more overhead.

11 Looking Ahead: Week 5

- Scheduling algorithms: first-come first-served (FCFS), shortest-job-first (SJF) and its preemptive form SRTF, priority scheduling, and round robin (RR).
- Gantt charts and average waiting and turnaround times for each algorithm, using exactly the criteria of this week.
- Process synchronisation: race conditions (our `shared_counter++`) and the critical-section problem.
- Mutex locks and semaphores.

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, Chapter 4 (Threads & Concurrency, Sections 4.1–4.4) and Chapter 5 (CPU Scheduling, Sections 5.1–5.2).
- W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed., Pearson, Chapters 4 and 9.
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Chapter 26, “Concurrency: An Introduction”: `https://pages.cs.wisc.edu/~remzi/OSTEP/threads-intro.pdf`
  - Chapter 27, “Interlude: Thread API”: `https://pages.cs.wisc.edu/~remzi/OSTEP/threads-api.pdf`
  - Chapter 7, “Scheduling: Introduction”: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf`